CANIS: building a conditioning robot a frightened dog can actually live with

CREST Gold Award project report

Supporting documents submitted alongside this report: `TIMELINE.md` (plan and time management) · `RISK_ASSESSMENT.md` (safety and welfare) · `../paper/canis_paper_draft_v2.md` (full systems paper) · `../BUILD_STORY.md` (build log) · `../PROBLEMS.md` (problem and risk inventory) · `../../hardware/TESTING.md` (dated bench-validation results).

> ***Scope, stated up front.*** *I'm submitting this as a* ***systems, design, and safety engineering*** *project. I'm making* ***no claim*** *that the device actually changes the dog's behaviour, and I didn't run a behavioural trial. That's a deliberate choice, and I explain why in §12: a behaviour-change claim would need a controlled, multi-subject, ethically-reviewed study, and reporting one without that would be the single worst thing I could do here. What I do have are engineering validation results, and they're real, dated, and logged.*
>
> ***Author:*** *Siddharth. No project mentor. Remaining self-checks are collected in §14.*

1. Aim and objectives

Aim: design and build a home robot that helps condition a dog's behaviour through consistent, precisely-timed reward, for a dog who's *afraid of moving objects*, which makes the robot itself the problem the whole design has to solve.

The subject is a young Goldendoodle who does several things I'd like him not to (eats things he shouldn't, barks, climbs the furniture, roams), and who's also measurably scared of movement. Most automated pet devices would make this dog's situation worse, not better. That one fact reframes the whole engineering problem: robot motion isn't a free actuator here, it's a **hazard I have to ration out carefully**.

Five objectives followed from that:

#	Objective	How I'd know	Outcome
O1	Deliver reward on a timescale that actually	A marker cue plus treat, triggered by a detected behaviour, with delivery	**Met.** Confirmed-ACK dispensing

#	Objective	How I'd know	Outcome
	matters for conditioning	confirmed rather than assumed	with correlation ids (§9)
O2	Never punish, only reward the alternative	No punishment path exists anywhere in the state machine	**Met by construction** (§3)
O3	Make robot motion safe by design, not by carefulness	The robot halts within ~1 s of losing its command link, in every failure mode I tested	**Met and bench-verified** (§8.4, §10)
O4	Make welfare a controllable, default-safe setting	A single explicit exposure level, defaulting to no motion, reversible, still useful at that default	**Met.** `EXPOSURE_LEVEL` (§8.5)
O5	Identify the dog reliably enough that false rewards are rare	Near-deterministic identification, no false triggers on other people or pets	**Met.** ArUco fiducial on the collar (§8.3), bench-verified end to end

A sixth objective, actually running the gradual-exposure protocol to the point of logged behavioural observations, **wasn't met**, and I explain why honestly in §11.

2. Wider purpose

Companion-animal behaviour problems are really common, and the reason most owners can't fix them isn't that they don't care, it's timing: proper conditioning needs consistent reinforcement within a second or two of the behaviour, every single time, and people just aren't that reliable. A device that's always there and always paying attention is genuinely appealing here.

The obvious risk is just as real, though. An automated "corrector" easily turns into something aversive and frightening, and for an animal that's already sensitive to movement, a machine that approaches whenever it disapproves is about the worst intervention you could design. The wider point of this project is showing those two things can actually be separated: you can

build a device that conditions behaviour *without* becoming something the animal is scared of.

The bigger, more general idea here is about where welfare actually lives in a system. In most designs it's spread thin, a bunch of good intentions scattered across the code, and any one of those can get quietly eroded by a later change without anyone noticing. CANIS concentrates it into one explicit, default-safe control instead (§8.5). I think that's a pattern that transfers to any system that acts physically on a living thing, and it's probably the part of this project that matters most to someone building something completely different.

3. Approaches I considered

Punishing entry, rejected on welfare grounds. The obvious design here, detect the dog in a zone he's not allowed in and deter him, is the standard approach, and I rejected it before writing any code at all. For a fearful dog, punishing zone entry pairs the zone *and the robot* with fear, which just gives you a dog who's now scared of a machine that lives in his house. What I went with instead is to **mark and reward the dog for leaving**: the behaviour I actually want gets reinforced, and disengaging is what earns the treat. Because the reward only ever fires on leaving, there's no punishment path in the system at all to accidentally misconfigure.

YOLO or a learned object detector, rejected on reliability. The instinctive choice for "find the dog" is a CNN. I went with fiducial markers instead, an ArUco `4x4_50` tag on the collar, for three reasons I argue in paper §3.6. The task here isn't "find any dog in any pose," it's "is *this specific* dog in *this specific* room," and a fiducial gives a near-deterministic, low-false-positive answer to that. That matters more here than it normally would, because **a false correction is actually harmful to a fearful animal**, so the cost of getting it wrong is lopsided in a way normal detection benchmarks don't account for. A CNN would also eat up way more of the Pi's compute for no real reliability gain, given there's only ever one cooperative, marker-wearing subject. And the marker basically *is* identity, so other people or pets in frame just don't trigger anything.

I want to be clear this is a pragmatic call, not a claim that fiducials generally beat learned detectors. For a reliability-critical, single-subject, compute-constrained deployment like this one, the fiducial is the right engineering choice. Change any of those three conditions and the answer probably changes too.

One single controller, rejected for failure isolation. Putting everything on one board is

simpler. I rejected it because it couples failure domains that really need to stay apart: a vision stall would delay motor commands, and a crash would take the safety watchdog down along with whatever it was watching. The three-tier split (§8.2) exists specifically so the real-time executor can never be starved by a network stall.

A printed chassis, abandoned partway through. After four CAD revisions, an available Meccano-style perforated metal chassis turned out to be simpler and stronger than the printed deck. I describe that switch in §11.

4. Plan and rationale

Full detail is in `TIMELINE.md`. The constraint *do no harm to a fearful animal* flipped the usual build order upside down: the stop path, the motion watchdog, and the exposure gate all had to exist and be tested **before** the robot was ever allowed near the dog at all. "Get it driving first, add safety later" was just never a plan I could actually use here.

Subsystems got validated on their own before integration, in the order set out in `hardware/TESTING.md`: motors, dispenser, sensors, the link between boards, vision, power, and only then the full loop. With three processors and two link protocols, tracking down a failure in the fully integrated system is expensive, and an unvalidated subsystem running near a frightened animal is a welfare risk, not just an ordinary bug.

5. Time management

The project ran from **June 26 to July 19, 2026**, 24 days, **84 hours** across seven phases, with an actual two-week gap between the public release (July 7) and the chassis pivot (July 19). `TIMELINE.md` has the full dated phase table and Gantt chart, traced back to commits and to the dated results log in `hardware/TESTING.md`.

Two things didn't go to plan:

CAD cost way more than I budgeted. Four revisions, each with a mechanical interference that only showed up once it was actually printed, which meant a full print cycle per attempt.

Phase 7 wasn't in the plan at all. The chassis pivot forced me to re-derive the whole mounting layout, wiring plan, and assembly order, since all of it had assumed a printed deck that no longer existed.

6. Resources and people

Three processors, each doing what it's actually good at. That's an unusual amount of hardware for a school project, and it's justified by failure isolation, not raw capability. An Arduino Uno as the real-time executor ("limbs") for jitter-free servo and motor timing, kept deliberately dumb and reliable on purpose. An ESP32 ("brain") running the arousal state machine and coordinating the other two tiers. A Raspberry Pi 5 ("eyes and face") for OpenCV vision and the Flask dashboard, which also keeps the wall clock.

Working with what I actually had. The best resource decision I made was giving up on four revisions of printed-chassis CAD once I realised a perforated metal chassis was simpler and stronger. That meant accepting that the castor's 24 mm bolt circle and the dispenser's three feet, at 25 mm radius and 120° apart, just don't land cleanly on a 10 mm grid. I solved it by bolting whichever foot happened to land on a real hole and zip-tying the rest, and I've recorded that honestly rather than pretending the grid and the CAD actually line up.

People, including being honest about the gap. This was a solo build, and the honest answer is that the expertise this project actually needs was *identified but never brought in*: paper §8 names prior review from an animal-research ethics body and/or a qualified veterinary behaviourist as something that would be required before making any efficacy claim. I haven't sought that review, which is exactly why the project is scoped to not make that claim at all. Knowing which expert you'd actually need, and scoping your claims to what you can support without them, feels like the responsible version of working alone rather than just pretending you didn't need help. I didn't use a mentor for this project. It was built independently the whole way through, and the profile form's mentor field is left blank for that reason.

The subject. The dog is a family pet, and everything he does here is bounded by supervision, the welfare gate, and an owner (me) who can end any session whenever I want (§12).

7. Background research

Three sources back up the specific claims I'm making; full citations in §16, and the related-work discussion is in paper §2.

The conditioning model here relies on established practice around using reward to shift a fear response *without flooding* the animal [1]. Flooding, forcing exposure to whatever's scaring the animal, is exactly what a badly-designed robot like this would end up doing by accident, and the gradual-exposure architecture (§8.5) exists specifically to avoid it. The staged, criterion-

gated shape of the exposure protocol follows the standardised training-program model in [2]. The perception layer uses the ArUco fiducial detection algorithm directly [3], which is also where the ArUco-over-YOLO argument in §3 comes from.

> ***Where this is thin (criterion 2.2):*** *three citations is fine but it's the thinner of my two projects, LYRA has six. Two more would genuinely help: something on canine fear-response and body-language coding, which the exposure protocol needs anyway to actually define its advance/hold/retreat criteria, and something on reinforcement timing and delay-of-reward, which relates directly to the ~5 to 9 second reward latency I've flagged as a validity risk in* `PROBLEMS.md` *§H.*

8. System design

8.1 Design constraints

Three constraints drove basically every decision after this (paper §3.1):

- **C1, do no harm to a fearful animal.** Motion is a hazard to be rationed. That makes "stops reliably and quickly" a safety requirement, and "works fine while not moving at all" a proper, first-class mode, not a fallback.
- **C2, reward leaving, never punish entering.**
- **C3, empirical, tunable thresholds**, named and kept in one place in both the firmware and the dashboard simulator, and documented as provisional rather than settled science.

8.2 Three-tier compute split

Device	Role	Why it's here
Arduino Uno, "limbs"	Real-time executor: motors, dispenser servo, marker buzzer	Jitter-free 5 V timing; kept deliberately dumb and reliable
ESP32, "brain"	Sensors, arousal state machine, decisions, both links	Enough compute for the state machine plus WiFi to coordinate everything
Raspberry Pi 5, "eyes and	OpenCV vision, Flask dashboard	Vision and a web UI need more than either

Device	Role	Why it's here
face"		MCU can give; the Pi keeps the wall clock

The split keeps failure domains separate. The real-time executor can't be starved by a network stall, and the decision tier *polls* the Pi instead of being driven by it synchronously, so vision dropping out just degrades the system instead of blocking it entirely.

8.3 Detection

Two independent sources feed into this. Locally, the ESP32 combines a PIR sensor with three ultrasonic rangefinders (left, centre, right), fired one after another to avoid echo cross-talk, which also gives a rough bearing. Remotely, the Pi picks up the collar's ArUco marker and posts a *zone label*, not any actual imagery, which the ESP32 then polls. Frame-skipping and a Laplacian-variance blur check reject motion-blurred frames instead of trying to classify them.

Ultrasonics give you range, not identity, so they genuinely can't tell the dog apart from a wall. That's exactly why, while the robot's moving, the dog is identified by the camera marker *only*, and I've recorded that constraint honestly in `PROBLEMS.md` §A instead of glossing over it.

8.4 The arousal state machine

A scalar **arousal** value between 0 and 100 drives three states, `CALM`, `AROUSAL`, and `CORRECTION`, with **rate-based hysteresis**: separate enter and exit thresholds, so arousal has to climb clearly past one line to switch a state on, and fall clearly past a lower line to switch it back off. Without hysteresis, a dog sitting right at the threshold made the state flip every 20 ms, which re-fired the buzzer dozens of times for one continuous event (§9).

Two numerical safeguards protect that integration. `dt` gets clamped to 0.2 s per tick, so a single stalled loop iteration can't make `rise-rate × dt` jump the system straight from calm to a full correction in one step. And the vision latch clears itself on a local 4-second timeout instead of waiting on a network round-trip (§9).

Fail-safe motion. The real-time Arduino runs a motion watchdog that halts within ~1 s of its command link going quiet. That means a crashed ESP32, a dead router, or a pulled cable all fail to *stopped*, never to *moving toward the dog*. Motors also stop on any reset, no exceptions.

8.5 The gradual-exposure gate: welfare boiled down to one knob

`EXPOSURE_LEVEL` puts welfare behind a single explicit control, described right there in the

code as "the welfare knob." It **defaults to 0, no movement at all**, and the system is still fully useful there, delivering marker and reward with zero motion. Levels only go up one at a time, only when the dog's logged fear response shows he's comfortable, and every step can be reversed. Going up a level needs a two-key arm-then-confirm instead of just one keypress, and exposure resets back to 0 on every boot.

The actual architectural point here is that this **separates what the robot decides from how much it's physically allowed to move.** I can develop, debug, and fully run the decision logic while the physical envelope stays completely at zero.

9. Problems I found and actually fixed

A latch that could never clear itself. The ESP32 decided "is the dog here?" partly from a vision flag stored in `visionActive`, which only ever got *cleared* by a successful poll coming back "no dog." If WiFi dropped, or the Pi crashed, or the router rebooted, that clearing poll never came back, and the latch just stuck on **forever**: the buzzer firing every two seconds, and at higher exposure levels the robot re-orienting toward a dog who might have wandered off an hour ago. This was genuinely the worst thing the system could do, and it was reachable by the single most common home-network failure there is. I fixed it with a local 4-second timeout that clears the latch with no network round-trip needed, and mirrored it in the test suite so it can't quietly regress later.

Chatter right at the threshold. A dog sitting exactly at the correction threshold made the state flip every 20 ms, re-firing the buzzer and inflating the intervention count dozens of times for what was really one continuous event. Fixed with hysteresis.

A safety guarantee that plain arithmetic could break. At the lowest "safe" exposure level, orient only, never approach, the bearing logic could legitimately calculate FORWARD as the direction to face the dog. The Arduino has no idea of "face" versus "drive at": FORWARD is just FORWARD. The one guarantee the whole project actually depends on could get broken by completely correct bearing arithmetic, not some rare-branch bug. Fixed by rewriting any FORWARD bearing below the approach-enabled level into an in-place pivot instead.

Treats getting counted that never actually fell. The dispenser counted a treat as given the moment the ESP32 *asked* for one. A jam or an empty hopper would still ring the marker tone and still bump the count up. For a system whose entire mechanism is "sound predicts treat," a marker with nothing behind it doesn't just waste one session, it actively teaches the dog the

marker is a lie. I fixed this two ways: delivery gets confirmed back from the Arduino so rewards only count on an actual ACK, not just the request, and a correlation id on each dispense means a coincidental manual button press can't get miscredited to a pending automatic reward. After two jams in a row, the dispenser goes quiet instead of continuing to beep over an empty hopper.

CAD that looked fine and didn't fit at all. Three print revisions each had a genuine interference: the treat chute ran straight through where the castor wheel needed to sit; the funnel's sloped wall clipped the shelf cutout; the castor spacer overlapped the deck skirt by 7 mm; the hopper lid was modelled as a straight plug pushed into a funnel that keeps narrowing, so it physically couldn't seat the last 2.5 mm. Every single one looked completely fine in a 3D preview and only broke once it was actual printed plastic, a full print cycle per discovery. The fix that actually mattered here is in §11.

10. Results: engineering validation

Bench tests, run per subsystem and logged. **Results log dated 2026-07-06** in `hardware/TESTING.md`:

Test	Result
1, Motors (L298N, both channels PWM-matched)	**PASS.** Forward, back, pivot, and stop all correct
2, Dispenser	**DEFERRED.** Mechanism works; `GATE_OPEN_MS` tuning happens once it's actually mounted, not on the bench
3, Sensors	**PASS.** All three ultrasonics plus PIR clean
4, ESP32 to Arduino link	**PASS**
5, Full three-way Pi to ESP32 to Arduino comms	**PASS.** Communication confirmed end to end

Also verified on real hardware (paper §4.1): vision working end to end, camera through OpenCV/ArUco detecting marker ID 0 through a directional zone posted to the dashboard, which the polling ESP32 picks up; the state machine's real outputs, including dispense, correction cue, reward cooldown and cap, and watchdog-safe pulsed orienting; and the

dashboard's live link with a simulator fallback that auto-starts on boot.

What I measured, and what I didn't. These are engineering-validation results: does each subsystem behave the way I said it would, does the closed loop actually make the transitions I intended, do the fail-safes fire when they're supposed to. **I didn't run a controlled behavioural trial**, and I didn't collect anything a behaviour-change claim would actually need, baseline behaviour rates, a control condition, blinded coding, multiple subjects. That's a deliberate decision about scope, not something I just forgot (§12).

11. Creative thinking

Flipping the conditioning approach on its head. The main creative decision in this whole project is realising the standard design isn't just not ideal for a fearful dog, it's actively counterproductive, and that flipping it, rewarding leaving instead of punishing entering, solves the welfare problem and the engineering problem at the exact same time. It removes the punishment path completely, so there's no aversive behaviour left anywhere in the system that could get triggered by a misconfiguration.

Turning welfare into a single number. Taking a vague design intention and turning it into one explicit, default-safe, reversible control is the part of this I think is most likely to actually transfer to something else. It turns "we were being careful" into a property you can actually inspect, test, default to safe, and reset on every boot.

Turning "does it fit" into an actual computation. The most useful method I came up with grew directly out of the CAD failures. Instead of just eyeballing a preview, I check every part's real assembly position by asking OpenSCAD to compute the literal 3D intersection between two placed solids: parts that don't actually touch intersect to nothing, and parts that overlap intersect to a solid shape you can see and measure. That turns "does this fit?" from a judgement call based on a render into a plain yes/no computation, and it now checks every dimension I claim anywhere in this project. The general idea matters more than the specific trick here: when your eye keeps getting a certain kind of question wrong, stop trying to look harder and find a way to actually *compute* the answer instead.

Actually naming the failure mode I was in. `PROBLEMS.md` records the project's honest top risk as *"build perfectionism means it never actually runs with the dog,"* right alongside the instruction to just ship the simplest working mode and go observe. Writing that down about my own project, in my own repo, was uncomfortable, and I think that's exactly why it makes

the rest of the document trustworthy. See §13.4.

12. Ethics and welfare

The full register is in `RISK_ASSESSMENT.md`, with animal-welfare hazards listed first because constraint C1 outranks everything else, capability included, everywhere in this system.

Scope. I'm not claiming CANIS changes the dog's behaviour. I didn't get institutional animal-ethics approval, on purpose, because I'm not reporting any behavioural result. If this work ever got reframed toward an efficacy claim, I'd need prior review from an animal-research ethics body and/or a qualified veterinary behaviourist *first*, and the exposure protocol's fear-response coding scheme would need to be pre-registered rather than defined after I'd already seen the data. Choosing the scope your evidence actually supports, instead of the scope that sounds most impressive, is itself an ethical decision, and I think it's the right one here.

Welfare-first design. Default no movement; a mild sound cue and a food reward; reinforcing a desired alternative rather than punishing anything; motion introduced one reversible step at a time, gated entirely on the dog's logged response.

Physical safety. A hardware motion watchdog that halts within ~1 s of any command-link loss; correction motions that are duration-bounded and stop immediately on disengagement; `NEVER_APPROACH` in every single mode; speeds turned down for exposure sessions; supervision the whole time, with the session ending immediately on any sign of distress **or** over-arousal. I watch for the second one just as closely as the first, since a robot that's being chased can wind a dog up instead of scaring him.

Reward safety. A 20 s cooldown and a 30-per-session cap, enforced in every mode with a compile-time guard, to stop over-feeding and hopper depletion. The treats are a vet-recommended training treat; how many get given varies with session length rather than being a fixed number, and the daily total is confirmed to sit comfortably inside a safe allowance.

Data and privacy. The camera only does on-device fiducial detection: a zone label gets posted, never any actual imagery. What gets recorded is sensor counters, arousal and state logs, and marker detections. The vision script has a `--snap` diagnostic flag that can save a still frame to disk; whether that's ever actually been used outside the one bring-up test, and what the retention policy is if so, is something I confirm myself in §14. The home network SSID turned up in tracked test sketches and got scrubbed, including from the audit file that had recorded it in the first place, and I've added a provenance convention so a stale claim like

that can't quietly drift unnoticed again.

Open welfare items, and I'm stating them as open. The beam-break jam detector has no mount and ships disabled, so a silent jam only gets caught by gate-cycle counting. Reward latency of ~5 to 9 seconds might reinforce whatever the dog happened to do in that gap; the immediate marker tone is the standard fix for that, but actually cutting the latency down is still open work. (Over-arousal used to be listed here too, but it's resolved now, since the stop-on-over-arousal rule is an explicit, confirmed protocol at this point, not a gap anymore.)

Consent. The animal's owner, me, consents to his taking part and can end any session whenever I want to.

13. Conclusions, what it means, and what I'd do differently

13.1 Conclusions

A conditioning robot for a fear-sensitive dog can genuinely be built so that welfare is an architectural property, not just something that depends on the operator being careful. Three specific conclusions come out of the validation in §10:

1. **Fail-safe motion is achievable on cheap hardware.** Splitting the real-time executor away from the decision tier, and giving the executor its own watchdog, means every failure path (crashed controller, dead network, a pulled cable) ends up at *stopped*. Verified on the bench.
2. **Welfare really can be reduced to one controllable number.** `EXPOSURE_LEVEL` separates decision-making from physical range, so the full decision logic can run and get debugged while the robot isn't allowed to move at all.
3. **A fiducial beats a learned detector under this particular cost trade-off.** When a false positive does real harm to a frightened animal, near-deterministic identification is worth more than generality, and it's also cheaper to run.

All five design objectives from §1 got met. The sixth one, actual behavioural observations, didn't, and I'm not making any behavioural claim anywhere in this report.

13.2 What this actually means

The transferable idea here is about **where welfare lives in a system that acts physically on a living thing.** Most engineering approaches spread care thin across a lot of separate decisions, and any one of those can get eroded by a later change without anyone noticing. Concentrating

it into one explicit, default-safe, reversible control that resets on boot makes it something you can actually inspect and test, and it means someone can check the safety property without reading through the entire codebase. That applies well past dogs: assistive robotics, care equipment, anything sharing physical space with someone who can't consent in the normal sense of the word.

The second idea is about cost asymmetry in detection. Most detection systems get benchmarked on symmetric metrics, but here a false positive actually harms the animal while a false negative just misses a training opportunity. When the costs are that lopsided, the right architecture might genuinely be the less fancy one.

13.3 How my decisions actually shaped what happened

Choosing reward-on-leaving over punishment decided everything that came after it. It's the reason there's no aversive path anywhere in the system to be reached by a misconfiguration, and it turned a welfare requirement into an actual structural property instead of a policy.

Building the stop path before the drive path is the reason the FORWARD-bearing violation got caught at all. The guarantee was explicit and actively being checked, so it got found by inspection, not because the robot did something visibly wrong near a frightened dog.

Choosing a fiducial over a CNN traded generality for determinism, and that was only the right trade because of the cost asymmetry involved. I wouldn't make the same call for a system where missing a detection was the expensive outcome instead.

Scoping the work as design rather than efficacy cost me the more impressive-sounding claim, but it's the reason every single number in this report can actually be defended.

Looking back, every one of these decisions traded a bigger claim or a more capable system for a property I could actually verify.

13.4 What I learnt, and what I'd do differently

The technical learning spanned three fields that don't usually overlap: real-time control systems (watchdogs, hysteresis, isolating failure domains, numerical safeguards on integration), computer vision (fiducial detection, blur rejection), and animal behaviour science (counter-conditioning, reward timing, the difference between reinforcing an alternative and just suppressing a behaviour).

The bigger lesson was about honesty as part of the engineering itself, not separate from it.

Two moments really taught me that. The dispenser counting treats that never actually fell was a case where dishonesty *inside* the system had a real cost outside it, a marker with nothing behind it teaches the dog the marker means nothing. And writing down that the project's biggest risk was my own perfectionism was genuinely uncomfortable, in exactly the way honest self-assessment usually is.

Four things I'd actually do to improve this, in order:

1. **Run the exposure protocol.** This is the real gap in the whole project. Level 0 is buzzer-only with zero motion, it was runnable from Phase 1 onward, and it needed nothing at all from the chassis. I built this as though the behavioural side depended on having the finished robot, and at Level 0 it just never did. I'd have weeks of logged fear-response data by now if I'd started it in parallel from the start.
2. **Define the fear-response coding scheme before collecting any data.** Latency to approach, tail and ear posture, avoidance distance, willingness to take the treat, with advance/hold/retreat criteria decided in advance. Defining any of that after seeing the data would make it worthless.
3. **Close the open welfare items.** Actually mount and enable the beam-break jam detector, and give over-arousal a real robot-side signal instead of relying only on me watching for it.
4. **Cut down reward latency.** At ~5 to 9 seconds, the marker tone is doing all the work of bridging that gap; shortening the loop itself would make the whole conditioning mechanism more solid.

14. Author checks before submission

Resolved: hours (84), mentor (none, §6), stop-session criteria (ends immediately on any sign of distress or over-arousal, §12), and treat budget (vet-recommended treat, daily total confirmed to fit safely, amount varies by session, §12).

Still genuinely open, and only I can close this one:

1. **Camera retention.** Confirm the `--snap` diagnostic has never actually written frames to disk outside the one bring-up test I've noted in §12, or state the retention policy if it has. This depends on how I've actually run the device, which no amount of reading the tracked launcher script is going to settle for me.
2. **Two more citations (§7), optional.** Canine fear-response coding and reward-delay

timing. Both would help the exposure protocol anyway, but this is a strengthening, not something blocking submission.

3. **Don't submit the paper draft with** `[Author TODO]` **markers still in it.** Paper §5 and §8 still carry unresolved placeholders. This report supersedes §5 by scoping to engineering validation instead; if the paper gets attached as a supporting document, resolve or remove those first, since an assessor reading a bracketed TODO reads it as abandoned work.

15. Criteria map

#	Criterion	Where it's covered
1.1	Clear aim, broken into objectives	§1, incl. the O1 to O5 table
1.2	Wider purpose	§2
1.3	Range of approaches identified	§3, four alternatives, each rejected on a stated reason
1.4	Plan described, with rationale	§4; `TIMELINE.md` §1
1.5	Time planned and organised well	§5; `TIMELINE.md` §2 to 4 (dated phases + Gantt)
2.1	Good use of materials and people	§6, including the honest account of expertise identified but not brought in
2.2	Research and referencing	§7, §16; paper §2
3.1	Logical conclusions + wider implications	§13.1, §13.2, conclusions drawn straight from §10's validation results
3.2	How actions and decisions affected the outcome	§13.3
3.3	What was learnt, and improvements	§13.4
4.1	Understanding of the science	§8, hysteresis, failure-domain isolation,

#	Criterion	Where it's covered
		numerical safeguards, fiducial detection, counter-conditioning
4.2	Ethical and safety decisions, incl. risk assessment	§12; `RISK_ASSESSMENT.md`
4.3	Creative thinking	§11
4.4	Problems identified and overcome	§9
4.5	Explained clearly	This report throughout

16. References

[1] S. R. Lindsay, *Handbook of Applied Dog Behavior and Training, Volume One: Adaptation and Learning*. Ames, IA: Iowa State University Press, 2000. ISBN 978-0-8138-0754-6.

[2] A. Stellato, S. Jajou, C. E. Dewey, T. M. Widowski, and L. Niel, "Effect of a Standardized Four-Week Desensitization and Counter-Conditioning Training Program on Pre-Existing Veterinary Fear in Companion Dogs," *Animals*, vol. 9, no. 10, p. 767, 2019. doi: 10.3390/ani9100767.

[3] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion," *Pattern Recognition*, vol. 47, no. 6, pp. 2280 to 2292, 2014. doi: 10.1016/j.patcog.2014.01.005.